



UNIVERSITÀ DEGLI STUDI DI PADOVA

Corso di Laurea in Informatica
Insegnamento di Ingegneria del Software

Anno Accademico 2025/2026

Specifica Tecnica

Architettura e Design di Sistema

Gruppo: Synergo Unit

Email: synergounit20@gmail.com

Versione: **v0.1.0**

Data: **YYYY-MM-DD**

Registro delle Modifiche

Versione	Data	Autore	Verificatore	Descrizione
0.0.0	YYYY-MM-DD	Nome Cognome	Nome Cognome	Prima stesura della struttura del documento.

Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Scopo del prodotto	6
1.3	Glossario	6
1.4	Riferimenti	6
1.4.1	Normativi	6
1.4.2	Informativi	6
2	Tecnologie Utilizzate	7
2.1	Linguaggi di programmazione	7
2.2	Framework	7
2.3	Librerie	7
2.4	Database	7
2.5	Strumenti di sviluppo	7
2.6	Motivazioni delle scelte tecnologiche	7
3	Architettura del Sistema	8
3.1	Architettura Generale	8
3.2	Architettura di Deployment	8
3.3	Architettura del Frontend	8
3.3.1	Gestione dello Stato e dell'Editor	8
3.3.2	Gestione Interazioni AI (Widget/Pannello AI)	8
3.4	Architettura del Backend	8
3.4.1	Livello di Presentazione / Controller	8
3.4.2	Livello di Business Logic (Core LLM)	8
3.4.3	Livello di Integrazione / Infrastructure	9
4	Interfaccia API e Comunicazione	10
4.1	Struttura dei Payload AI	10
4.2	Gestione degli Errori e Fallback	10
5	Design Pattern Utilizzati	11
5.1	Pattern Creazionali	11
5.1.1	[Inserire Pattern Creazionale, es. Factory / Dependency Injection]	11
5.2	Pattern Strutturali	11
5.2.1	[Inserire Pattern Strutturale, es. Adapter]	11
5.3	Pattern Comportamentali	11

5.3.1	[Inserire Pattern Comportamentale, es. Strategy]	11
5.3.2	[Inserire Pattern Comportamentale, es. State / Observer]	11
6	Flusso dei Dati (Data Flow)	12
6.1	Flusso di esecuzione: Elaborazione AI (es. Distant Writing o Analisi)	12
7	Tracciamento e Stato dei Requisiti	13
7.1	Tabella Tracciamento Componenti-Requisiti	13
7.2	Stato di Soddisfacimento dei Requisiti	13

Elenco delle figure

Elenco delle tabelle

1	Stato dei Requisiti	13
---	-------------------------------	----

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di definire l'architettura di base e di dettaglio del sistema **Second Brain**. Vengono illustrate le scelte tecnologiche effettuate dal gruppo Synergo Unit durante la fase di Product Baseline, i pattern architetturali e di design adottati, la decomposizione in componenti del sistema e il tracciamento bidirezionale tra queste componenti e i requisiti individuati nella fase di Analisi.

1.2 Scopo del prodotto

[Breve riepilogo dello scopo del prodotto, analogo a quanto inserito negli altri documenti, focalizzato su gestione della conoscenza personale, LLM e integrazione Markdown]

1.3 Glossario

Al fine di evitare ambiguità terminologiche, i termini tecnici, gli acronimi e le parole che potrebbero avere interpretazioni multiple sono definiti nel documento [Glossario](#). Tali termini sono contrassegnati nel testo con il pedice _G.

1.4 Riferimenti

1.4.1 Normativi

- [Capitolato C6 – Second Brain \(Zucchetti S.p.A.\)](#);
- [Norme di Progetto, vX.X.X](#);

1.4.2 Informativi

- [Analisi dei Requisiti, vX.X.X](#);
- [Piano di Qualifica, vX.X.X](#);

2 Tecnologie Utilizzate

2.1 Linguaggi di programmazione

[Descrizione dei linguaggi adottati per frontend e backend]

2.2 Framework

[Descrizione dei framework principali]

2.3 Librerie

[Elenco delle librerie essenziali, es. per il parsing Markdown, chiamate HTTP, ecc.]

2.4 Database

[Descrizione della soluzione di persistenza scelta]

2.5 Strumenti di sviluppo

[Toolchain di sviluppo e build]

2.6 Motivazioni delle scelte tecnologiche

[Analisi di pregi e difetti per giustificare le adozioni tecnologiche in relazione ai requisiti del capitolato e alle risorse del gruppo]

3 Architettura del Sistema

3.1 Architettura Generale

Il sistema *Second Brain* è basato su un'architettura di tipo Client-Server per garantire la separazione delle responsabilità (Separation of Concerns) tra la gestione dell'interfaccia utente (editor Markdown) e l'orchestrazione delle chiamate ai Modelli di Linguaggio di Grande Scala (LLM). [Inserire diagramma architettura di alto livello]

3.2 Architettura di Deployment

[Descrizione della strategia di containerizzazione scelta (es. Docker). Specificare come vengono deployati il Frontend, il Backend e l'eventuale strato di persistenza dati/database].

3.3 Architettura del Frontend

Il Frontend gestisce l'interazione dell'utente con l'editor Markdown e la visualizzazione dello stato delle operazioni AI. [Descrivere il pattern architetturale scelto per il frontend, es. MVVM, Architettura a Componenti, ecc.]

3.3.1 Gestione dello Stato e dell'Editor

[Descrivere come il sistema mantiene lo stato dell'editor (testo inserito, renderizzazione Markdown) e come viene garantita la sincronizzazione dei dati per l'auto-salvataggio o la persistenza locale].

3.3.2 Gestione Interazioni AI (Widget/Pannello AI)

[Descrivere l'architettura dei componenti responsabili di inviare il prompt, mostrare lo stato di caricamento (es. spinner o streaming del testo) e permettere l'inserimento/scarto della risposta dell'LLM].

3.4 Architettura del Backend

Il Backend funge da orchestratore centrale e da strato di astrazione verso i provider LLM esterni. [Descrivere il pattern architetturale scelto per il backend, es. Architettura a Livelli (Layered), Architettura Esagonale (Ports and Adapters), ecc.]

3.4.1 Livello di Presentazione / Controller

[Descrizione di come il backend riceve le richieste dal client relative a operazioni testuali o di AI].

3.4.2 Livello di Business Logic (Core LLM)

[Descrizione del motore centrale: come il backend costruisce dinamicamente i prompt, come gestisce le logiche dei "Sei Cappelli di De Bono", del "Distant Writing" e della traduzione].

3.4.3 Livello di Integrazione / Infrastructure

[Descrizione di come il backend maschera le dipendenze specifiche dei provider LLM (es. OpenAI, modelli locali) e gestisce la formattazione delle risposte per il client].

4 Interfaccia API e Comunicazione

Al fine di garantire il disaccoppiamento tra l'editor Markdown (Frontend) e l'elaboratore di prompt (Backend), la comunicazione avviene tramite [specificare il paradigma scelto, es. REST, RPC, GraphQL].

4.1 Struttura dei Payload AI

La comunicazione relativa alle operazioni di Intelligenza Artificiale (Riassunto, Traduzione, Riscrittura, Sei Cappelli, Distant Writing) richiede una struttura dati definita. [Descrivere in dettaglio le interfacce o i DTO (Data Transfer Objects) utilizzati per la comunicazione client-server. Es. struttura della richiesta contenente operazione, testo contesto e prompt; struttura della risposta contenente esito e testo generato].

4.2 Gestione degli Errori e Fallback

[Descrivere come vengono gestite, normalizzate e comunicate all'utente le anomalie di rete, i timeout dovuti all'LLM e le risposte malformate].

5 Design Pattern Utilizzati

Di seguito sono illustrati i design pattern adottati per risolvere problematiche ricorrenti e garantire la scalabilità del codice di *Second Brain*.

5.1 Pattern Creazionali

5.1.1 [Inserire Pattern Creazionale, es. Factory / Dependency Injection]

[Spiegare come il pattern viene usato per istanziare dinamicamente le strategie operative o i client LLM in base alla configurazione].

5.2 Pattern Strutturali

5.2.1 [Inserire Pattern Strutturale, es. Adapter]

[Spiegare come il pattern viene utilizzato per mascherare le differenze tra le varie API esterne dei provider LLM, offrendo un'interfaccia unica al sistema].

5.3 Pattern Comportamentali

5.3.1 [Inserire Pattern Comportamentale, es. Strategy]

[Spiegare come il pattern isola la logica di costruzione dei prompt per le diverse operazioni (Traduzione, Analisi Sei Cappelli, Riassunto), permettendo di aggiungere nuove funzionalità senza modificare il flusso principale].

5.3.2 [Inserire Pattern Comportamentale, es. State / Observer]

[Spiegare come viene gestita la reattività della UI nel Frontend durante l'attesa di una risposta dall'LLM (es. stati di Idle, Loading, Success, Error) o la gestione degli eventi dell'editor Markdown].

6 Flusso dei Dati (Data Flow)

Questa sezione illustra il ciclo di vita delle informazioni all'interno del sistema, partendo dall'interazione dell'utente fino all'ottenimento del risultato generato dall'Intelligenza Artificiale.

6.1 Flusso di esecuzione: Elaborazione AI (es. Distant Writing o Analisi)

[Inserire un diagramma di sequenza UML o di attività]

1. **Interazione Utente (Frontend):** L'utente seleziona una porzione di testo nell'editor Markdown e richiede un'operazione specifica (es. "Riscrivi in stile formale").
2. **Creazione Richiesta:** Il Frontend impacchetta il contesto e il comando e lo invia al Backend tramite chiamata API.
3. **Validazione e Costruzione Prompt (Backend):** Il Backend riceve la richiesta, determina la strategia appropriata e costruisce un prompt strutturato (System + User context).
4. **Chiamata Esterna:** Il Backend contatta il servizio LLM esterno tramite l'apposito Adapter.
5. **Parsing e Restituzione:** Il Backend riceve il testo, esegue operazioni di parsing (es. rimozione metadati o tag Markdown spuri) e restituisce il payload normalizzato al Frontend.
6. **Aggiornamento UI:** L'interfaccia mostra la proposta all'utente, permettendone l'accettazione, il rifiuto o la rigenerazione.

7 Tracciamento e Stato dei Requisiti

Al fine di garantire che l'architettura progettata e le tecnologie scelte soddisfino le funzionalità concordate, si fornisce di seguito lo stato di implementazione dei requisiti emersi in fase di Analisi.

7.1 Tabella Tracciamento Componenti-Requisiti

[Inserire tabelle o riferimenti su come le singole componenti o i package descritti in architettura vadano a coprire i requisiti del capitolato].

7.2 Stato di Soddisfacimento dei Requisiti

La tabella seguente riassume lo stato finale dei requisiti funzionali e prestazionali richiesti dall'appalto.

Tabella 1: Stato dei Requisiti

Codice	Descrizione	Tipo	Stato
RO-F-1	Il sistema deve fornire un'area per l'editing di testo con supporto Markdown.	Obbligatorio	Soddisfatto
RO-F-2	Il sistema deve permettere la generazione di testo tramite LLM (Distant Writing).	Obbligatorio	Soddisfatto
RO-F-3	Il sistema deve permettere l'analisi del testo secondo i "Sei Cappelli di De Bono".	Obbligatorio	Soddisfatto
RO-V-1	Il sistema deve operare su un'architettura client-server accessibile via web.	Vincolo	Soddisfatto